

ASSIGNMENT NO. 12: MINI PROJECT – TRAINING INSTITUTE MANAGEMENT SYSTEM

Objective:

Develop a basic **MySQL Training Institute Management System** to manage students, trainers, courses, batches, enrollments, attendance, and assessment results. The project should demonstrate database planning, table relationships, query writing, business analysis, and presentation of findings.

Requirement Analysis and Project Planning

- Study how a training institute manages students, trainers, courses, batches, attendance, and assessment results.
- Identify the main users of the system, such as administrator, trainer, and institute coordinator.
- Define the information that must be stored for each business activity.
- Prepare a short list of functional requirements.
- Define the main reports expected from the database.
- Select suitable sample data representing a real training institute.

Database Design and Table Relationships

- Identify the main tables required for the project.
- Create a basic ER diagram showing how the tables are connected.
- Assign a primary key to every table.
- Use foreign keys to establish relationships.
- Avoid storing repeated student, trainer, course, or batch information.
- Ensure that every enrollment, attendance record, and assessment result is connected to valid records.

Required Table Relationships

Parent Table	Related Table	Relationship Type	Purpose
Course	Batch	One-to-Many	One course can be conducted through multiple batches

Trainer	Batch	One-to-Many	One trainer can handle multiple batches
Student	Enrollment	One-to-Many	One student can have multiple enrollments
Batch	Enrollment	One-to-Many	One batch can contain multiple student enrollments
Parent Table	Related Table	Relationship Type	Purpose
Enrollment	Attendance	One-to-Many	One enrollment can have multiple attendance records
Enrollment	AssessmentResult	One-to-Many	One enrolled student can receive multiple assessment results

Main Tables to Use

- Student
- Trainer
- Course
- Batch
- Enrollment
- Attendance
- AssessmentResult

Database Creation and Sample Data Preparation

- Create the project database in MySQL.
- Create all required tables according to the approved ER diagram.
- Apply suitable primary-key, foreign-key, and NOT NULL constraints.
- Insert realistic sample records into every table.
- Include multiple courses, trainers, batches, students, attendance records, and assessment results.
- Ensure that the sample data supports meaningful analysis.
- Verify the inserted data using basic SELECT queries.
- Check whether all foreign-key relationships are working correctly.

Business Queries and Data Analysis

Write MySQL queries to answer business questions such as:

- Display all students enrolled in each batch.
- Display the course and trainer assigned to every batch.
- Find the total number of students in each course.
- Find batches having more than a specified number of students.
- Display students whose attendance is below the required percentage.
- Find the average assessment score for each batch.
- Identify the highest-performing student in every batch.
- Display students who have not attended any session.
- Compare course-wise student performance.
- Rank students according to assessment results.
- Find batches with the highest and lowest average attendance.
- Generate a student-wise training progress report.
- Use joins, aggregate functions, subqueries, views, and window functions wherever suitable.

Insights, Documentation, and Final Presentation

- Summarize the important findings obtained from the queries.
- Identify courses with the highest student enrollment.
- Identify batches with low attendance.
- Identify high-performing and low-performing students.
- Explain how the institute can use the results to improve training.
- Present the ER diagram and table relationships.
- Present selected business questions and their query logic.
- Display query outputs in clear table format.
- Explain each query briefly without reading the complete SQL statement.
- Submit the MySQL script containing database creation, sample data, and queries.
- Submit a short project report containing the objective, relationships, analysis, insights, and conclusion.

- Objective: Develop a MySQL Training Institute Management System for students, trainers, courses, batches, enrollments, attendance and assessment results. The workbook requires an ER design, keys, sample data, business queries, analysis and a short report.

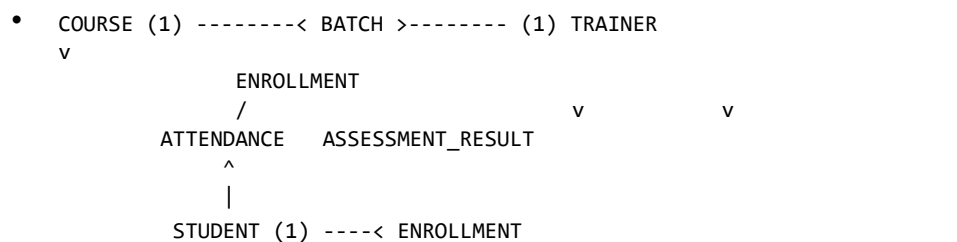
1. Requirement Analysis

User	Main work
Administrator	Manage students, trainers, courses, batches and reports
Trainer	View assigned batches, attendance and assessment results
Coordinator	Manage enrollments and monitor training progress

- Functional requirements: maintain student/trainer/course/batch data; enroll students; record attendance; record assessments; generate performance and attendance reports.

2. ER Diagram / Table Relationships

- Text ER representation:



- Required relationships are one-to-many: Course→Batch, Trainer→Batch, Student→Enrollment, Batch→Enrollment, Enrollment→Attendance and Enrollment→AssessmentResult.

3. Complete MySQL Database Script

```

CREATE DATABASE TrainingInstituteDB;
USE TrainingInstituteDB;
  
```

```

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(60) NOT NULL,
  Email VARCHAR(80) UNIQUE NOT NULL,
  City VARCHAR(40)
);
  
```

```

CREATE TABLE Trainer (
  TrainerID INT PRIMARY KEY,
  TrainerName VARCHAR(60) NOT NULL,
  Specialization VARCHAR(60) NOT NULL
);
  
```

```

CREATE TABLE Course (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(60) NOT NULL,
  
```

```

    DurationWeeks INT NOT NULL,
    Fees DECIMAL(10,2) NOT NULL
);

CREATE TABLE Batch (
    BatchID INT PRIMARY KEY,
    BatchName VARCHAR(40) NOT NULL,
    CourseID INT NOT NULL,
    TrainerID INT NOT NULL,
    StartDate DATE NOT NULL,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID),
    FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID) );

CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    BatchID INT NOT NULL,
    EnrollmentDate DATE NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (BatchID) REFERENCES Batch(BatchID)
);

CREATE TABLE Attendance (
    AttendanceID INT PRIMARY KEY,
    EnrollmentID INT NOT NULL,
    AttendanceDate DATE NOT NULL,
    Status ENUM('Present','Absent') NOT NULL,
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);

CREATE TABLE AssessmentResult (
    AssessmentID INT PRIMARY KEY,
    EnrollmentID INT NOT NULL,
    AssessmentName VARCHAR(50) NOT NULL,
    Score DECIMAL(5,2) NOT NULL,
    AssessmentDate DATE NOT NULL,
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);

INSERT INTO Student VALUES
(1,'Aarav Shah','aarav@gmail.com','Pune'),
(2,'Priya Patil','priya@gmail.com','Mumbai'),
(3,'Rohan Desai','rohan@gmail.com','Pune'),
(4,'Sneha Joshi','sneha@gmail.com','Nashik'),
(5,'Karan Mehta','karan@gmail.com','Thane'),
(6,'Neha Kulkarni','neha@gmail.com','Pune'),
(7,'Aman Verma','aman@gmail.com','Mumbai'),
(8,'Isha Nair','isha@gmail.com','Pune');

INSERT INTO Trainer VALUES
(1,'Rahul Kulkarni','Python & Data Science'),
(2,'Sneha Patil','SQL & DBMS'),
(3,'Vikram Joshi','Web Development');

INSERT INTO Course VALUES
(101,'Data Science',16,30000),
(102,'Database Management',12,22000),
(103,'Web Development',14,25000);

INSERT INTO Batch VALUES
(201,'DS-Morning',101,1,'2026-06-01'),

```

```
(202,'DBMS-Evening',102,2,'2026-06-05'),
(203,'Web-Morning',103,3,'2026-06-10'),
(204,'DS-Evening',101,1,'2026-07-01');
```

```
INSERT INTO Enrollment VALUES
```

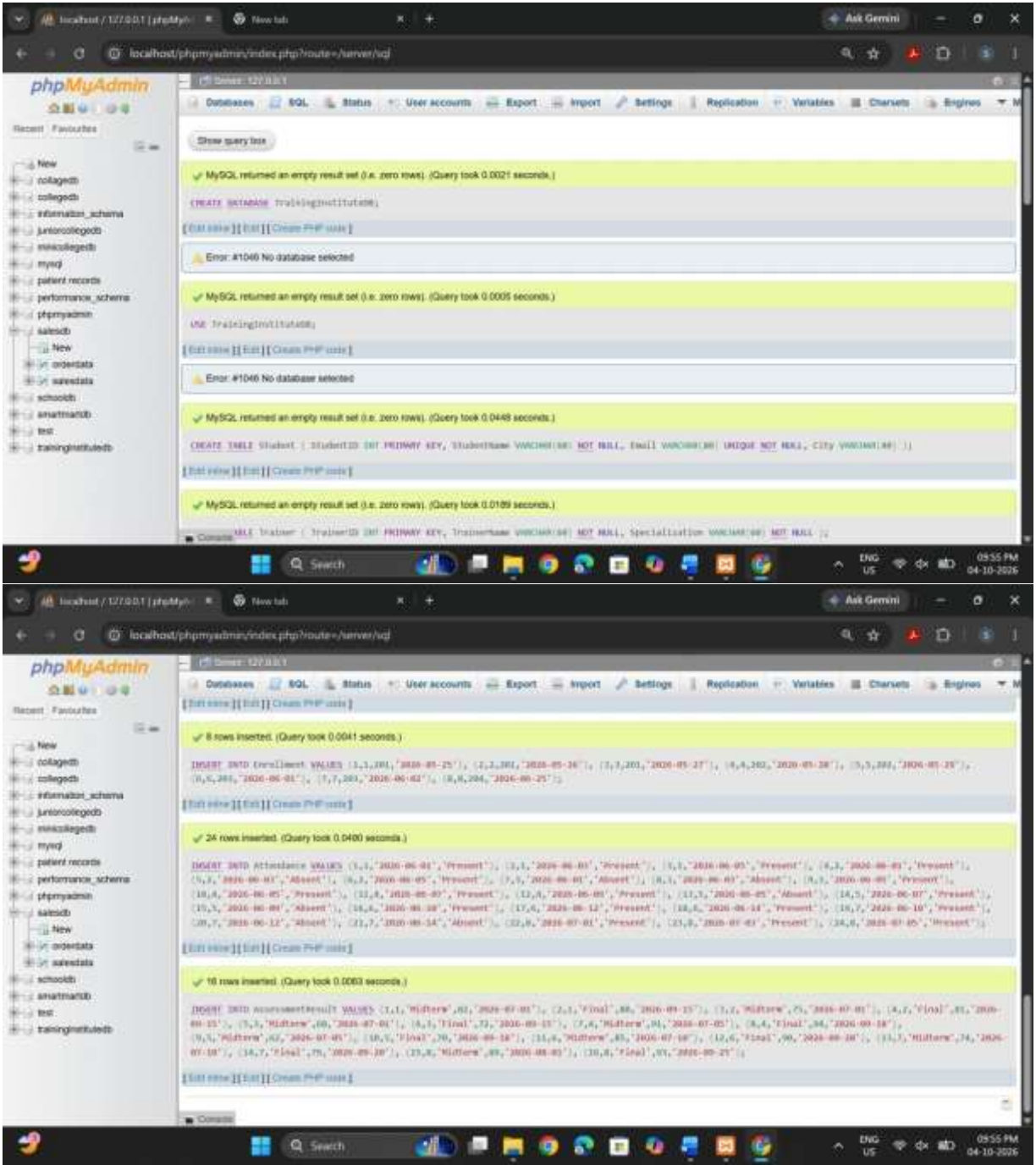
```
(1,1,201,'2026-05-25'),
(2,2,201,'2026-05-26'),
(3,3,201,'2026-05-27'),
(4,4,202,'2026-05-28'),
(5,5,202,'2026-05-29'),
(6,6,203,'2026-06-01'),
(7,7,203,'2026-06-02'),
(8,8,204,'2026-06-25');
```

```
INSERT INTO Attendance VALUES
```

```
(1,1,'2026-06-01','Present'),
(2,1,'2026-06-03','Present'),
(3,1,'2026-06-05','Present'),
(4,2,'2026-06-01','Present'),
(5,2,'2026-06-03','Absent'),
(6,2,'2026-06-05','Present'),
(7,3,'2026-06-01','Absent'),
(8,3,'2026-06-03','Absent'),
(9,3,'2026-06-05','Present'),
(10,4,'2026-06-05','Present'),
(11,4,'2026-06-07','Present'),
(12,4,'2026-06-09','Present'),
(13,5,'2026-06-05','Absent'),
(14,5,'2026-06-07','Present'),
(15,5,'2026-06-09','Absent'),
(16,6,'2026-06-10','Present'),
(17,6,'2026-06-12','Present'),
(18,6,'2026-06-14','Present'),
(19,7,'2026-06-10','Present'),
(20,7,'2026-06-12','Absent'),
(21,7,'2026-06-14','Absent'),
(22,8,'2026-07-01','Present'),
(23,8,'2026-07-03','Present'),
(24,8,'2026-07-05','Present');
```

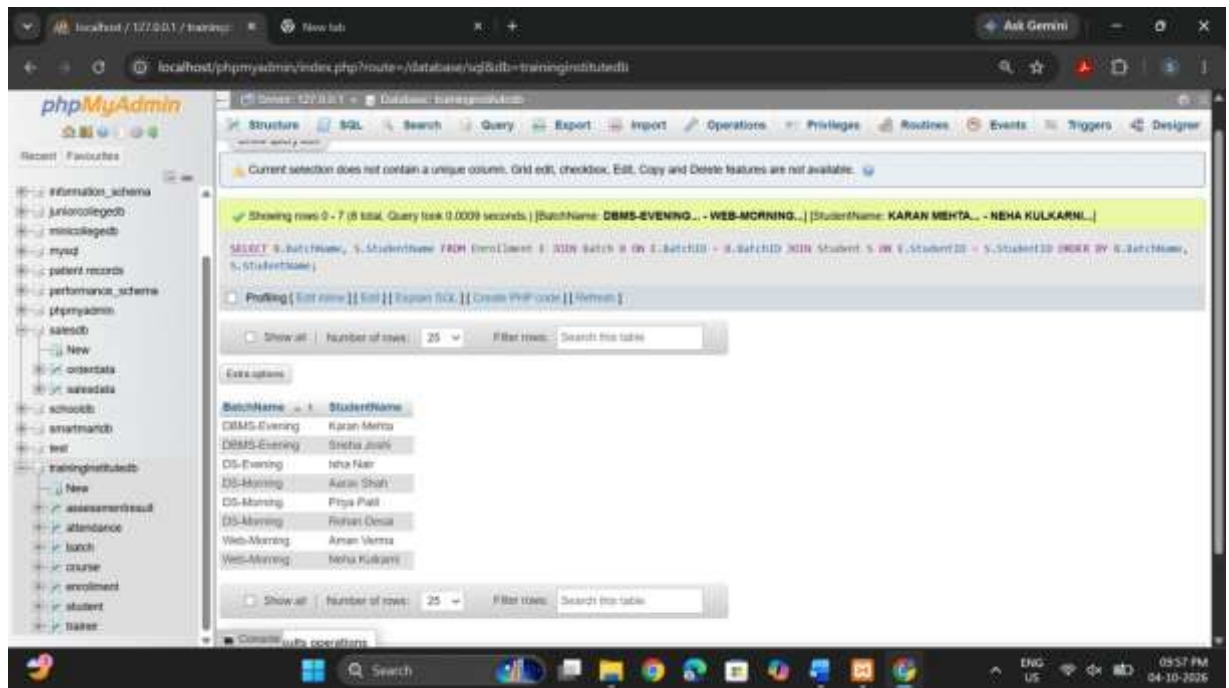
```
INSERT INTO AssessmentResult VALUES
```

```
(1,1,'Midterm',82,'2026-07-01'), (2,1,'Final',88,'2026-09-15'),
(3,2,'Midterm',75,'2026-07-01'), (4,2,'Final',81,'2026-09-15'),
(5,3,'Midterm',68,'2026-07-01'), (6,3,'Final',72,'2026-09-15'),
(7,4,'Midterm',91,'2026-07-05'), (8,4,'Final',94,'2026-09-18'),
(9,5,'Midterm',62,'2026-07-05'),
(10,5,'Final',70,'2026-09-18'),
(11,6,'Midterm',85,'2026-07-10'), (12,6,'Final',90,'2026-09-20'),
(13,7,'Midterm',74,'2026-07-10'), (14,7,'Final',79,'2026-09-20'),
(15,8,'Midterm',89,'2026-08-01'),
(16,8,'Final',93,'2026-09-25');
```



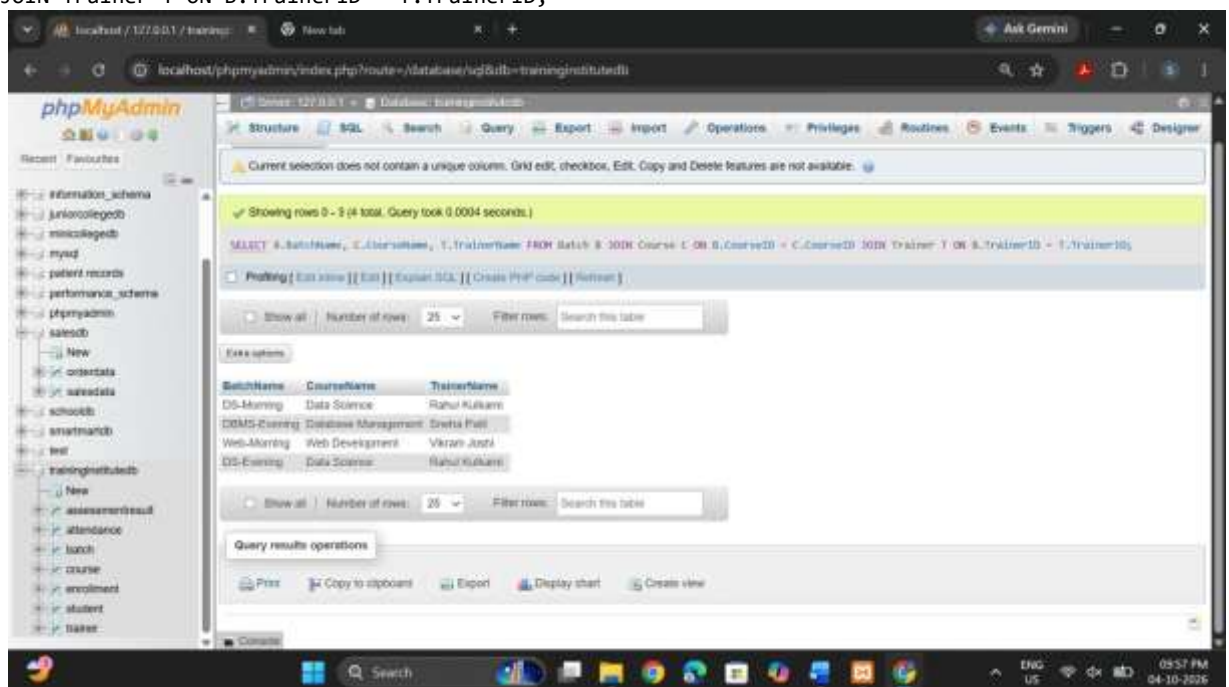
- 4. Business Query 1 — Display all students enrolled in each batch.

```
SELECT B.BatchName, S.StudentName
FROM Enrollment E
JOIN Batch B ON E.BatchID = B.BatchID
JOIN Student S ON E.StudentID = S.StudentID
ORDER BY B.BatchName, S.StudentName;
```



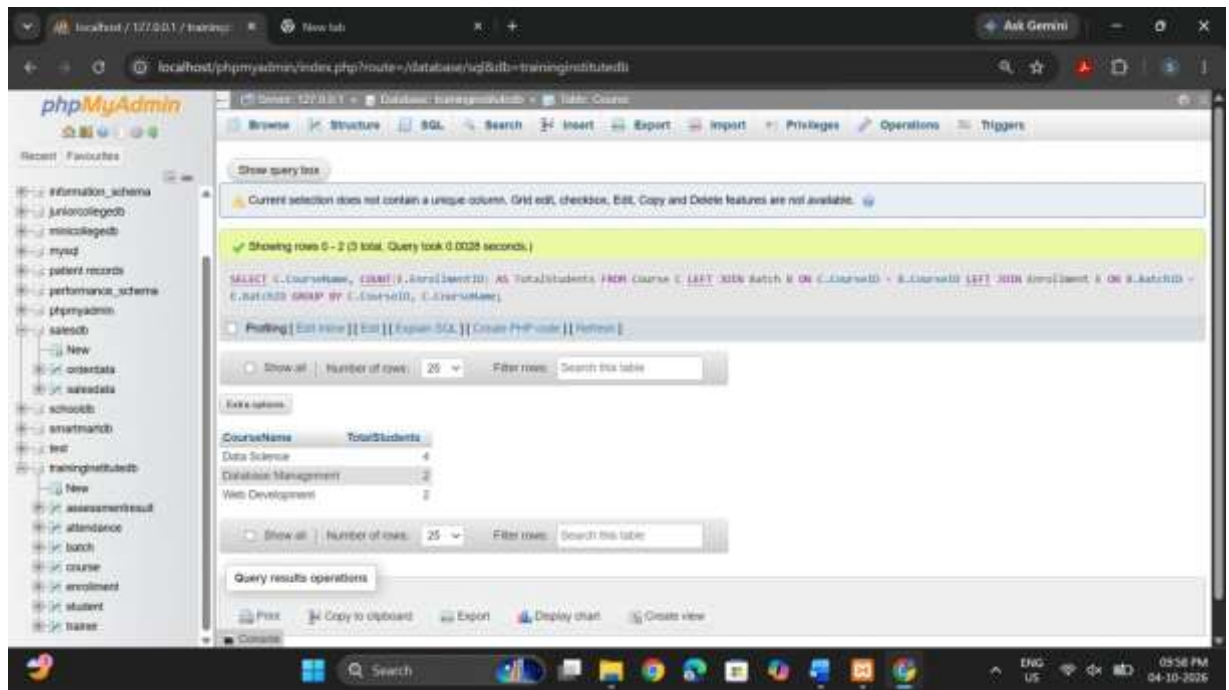
- 5. Business Query 2 — Display course and trainer assigned to every batch.

```
SELECT B.BatchName, C.CourseName, T.TrainerName
FROM Batch B
JOIN Course C ON B.CourseID = C.CourseID
JOIN Trainer T ON B.TrainerID = T.TrainerID;
```



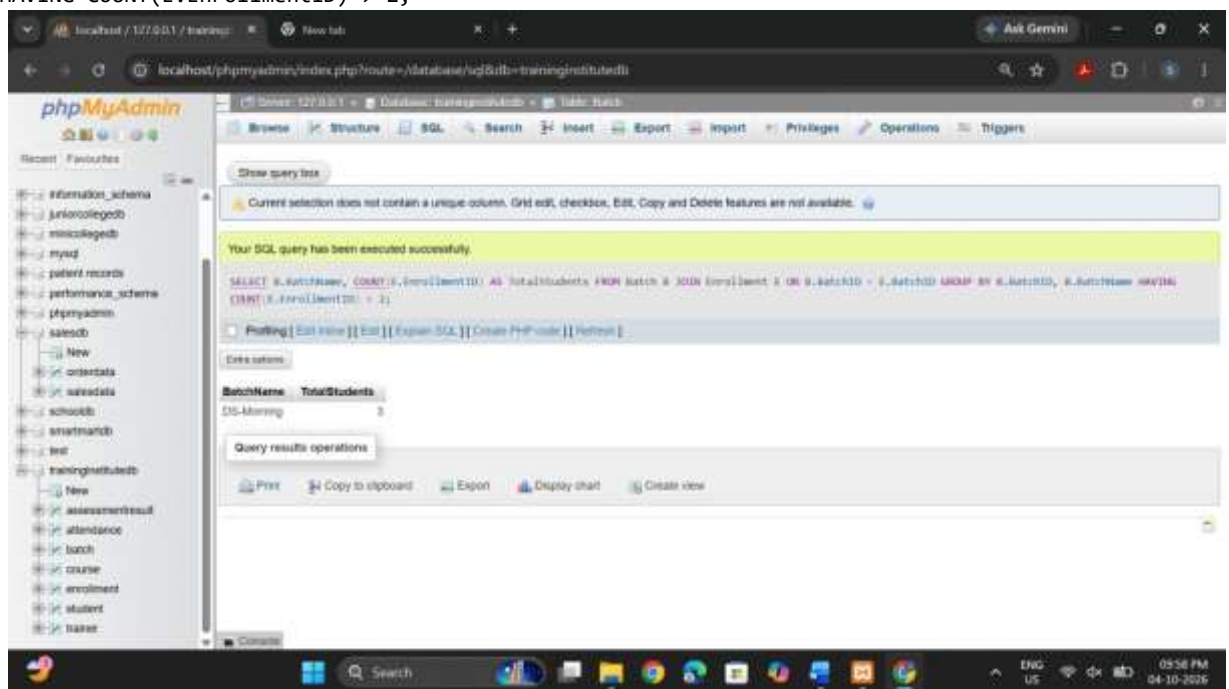
- 6. Business Query 3 — Total students in each course.

```
SELECT C.CourseName, COUNT(E.EnrollmentID) AS TotalStudents
FROM Course C
LEFT JOIN Batch B ON C.CourseID = B.CourseID
LEFT JOIN Enrollment E ON B.BatchID = E.BatchID
GROUP BY C.CourseID, C.CourseName;
```



7. Business Query 4 — Batches having more than 2 students.

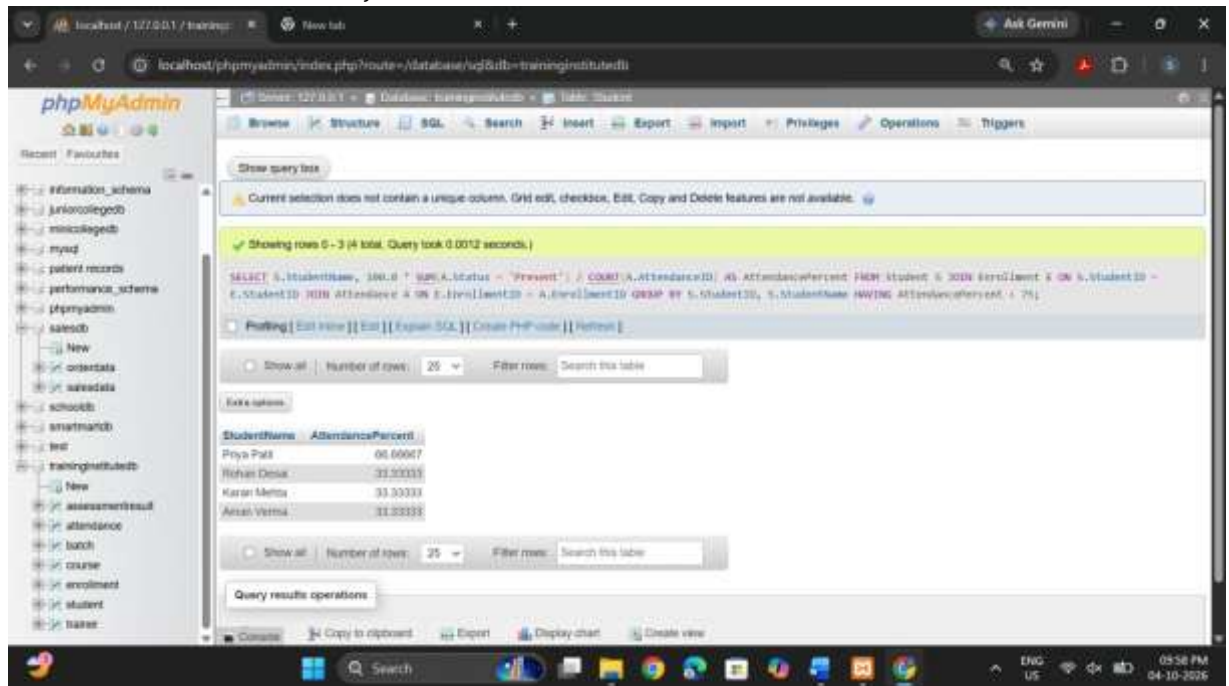
```
SELECT B.BatchName, COUNT(E.EnrollmentID) AS TotalStudents
FROM Batch B
JOIN Enrollment E ON B.BatchID = E.BatchID
GROUP BY B.BatchID, B.BatchName
HAVING COUNT(E.EnrollmentID) > 2;
```



8. Business Query 5 — Students below 75% attendance.

```
SELECT S.StudentName,
       100.0 * SUM(A.Status = 'Present') / COUNT(A.AttendanceID) AS AttendancePercent
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID
GROUP BY S.StudentID, S.StudentName
```

HAVING AttendancePercent < 75;



The screenshot shows the phpMyAdmin interface with the 'traininginstitute' database selected. A SQL query is entered in the query box, and the results are displayed in a table below.

SQL Query:

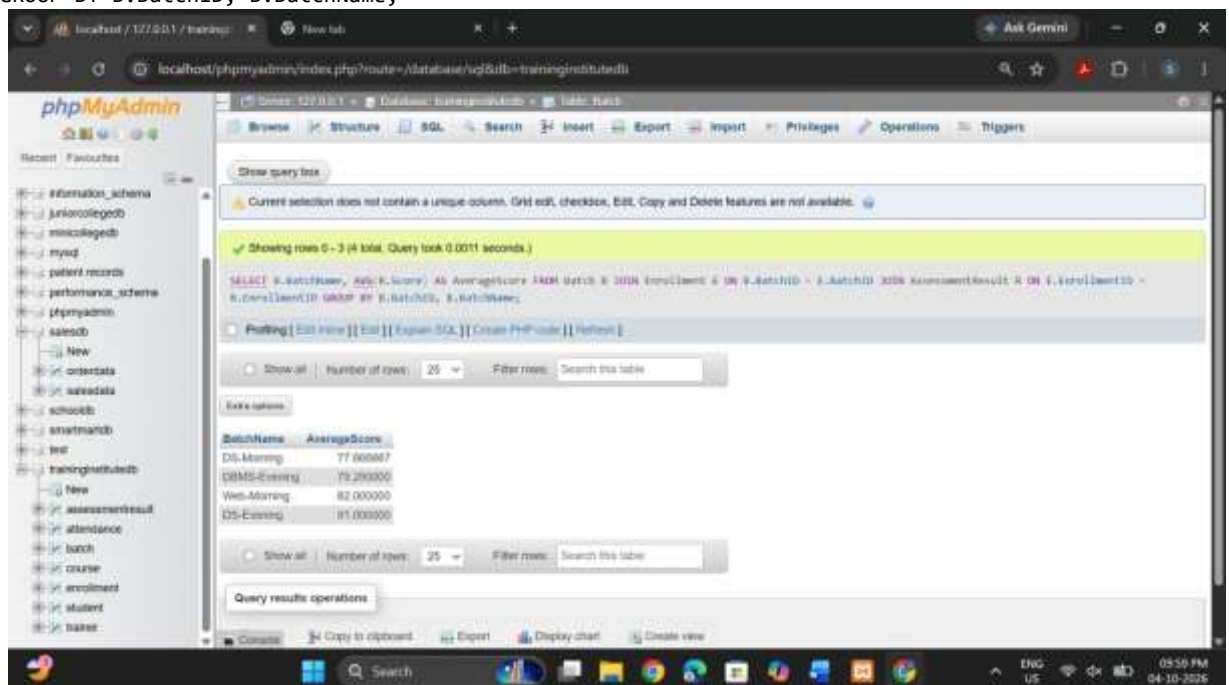
```
SELECT S.StudentName, SUM(A.AttendancePercent) AS AttendancePercent FROM Student S JOIN Enrollment E ON S.StudentID = E.StudentID JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID GROUP BY S.StudentID, S.StudentName HAVING AttendancePercent < 75;
```

Query Results:

StudentName	AttendancePercent
Priya Patel	66.66667
Rohan Desai	33.33333
Karan Mehta	33.33333
Anish Verma	33.33333

9. Business Query 6 — Average assessment score for each batch.

```
SELECT B.BatchName, AVG(R.Score) AS AverageScore
FROM Batch B
JOIN Enrollment E ON B.BatchID = E.BatchID
JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID
GROUP BY B.BatchID, B.BatchName;
```



The screenshot shows the phpMyAdmin interface with the 'Batch' table selected. A SQL query is entered in the query box, and the results are displayed in a table below.

SQL Query:

```
SELECT B.BatchName, AVG(R.Score) AS AverageScore FROM Batch B JOIN Enrollment E ON B.BatchID = E.BatchID JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID GROUP BY B.BatchID, B.BatchName;
```

Query Results:

BatchName	AverageScore
DS-Morning	77.000000
DBMS-Evening	79.200000
Web-Morning	82.000000
DS-Evening	81.000000

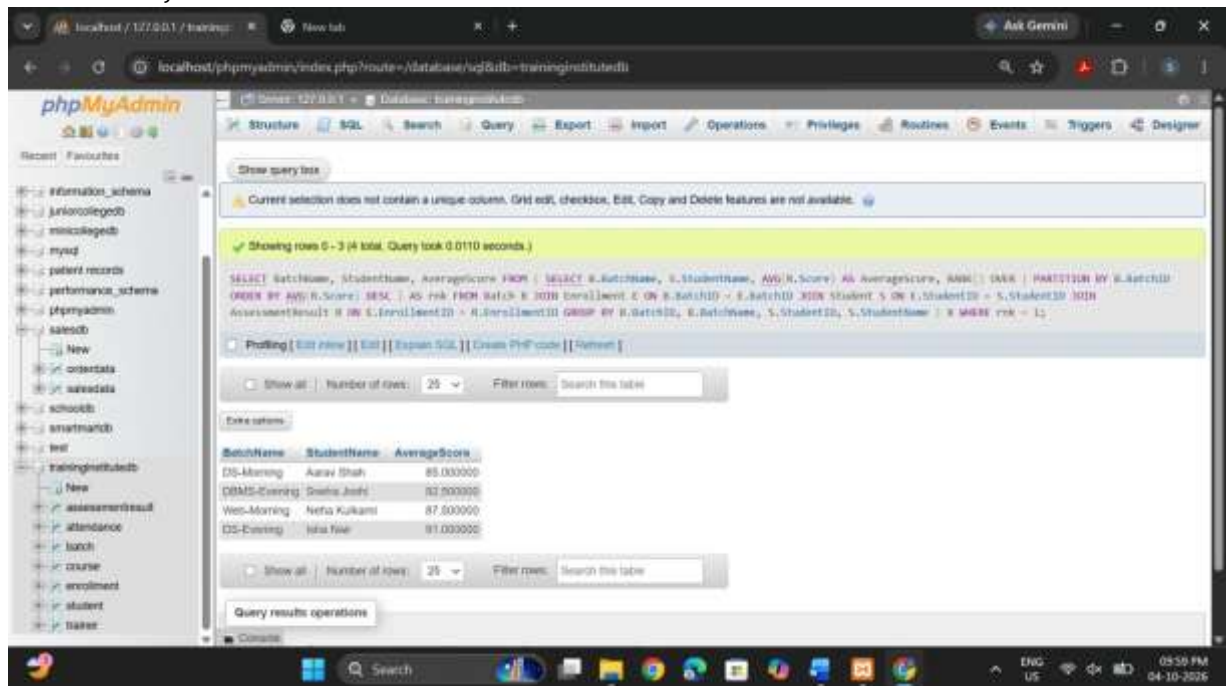
10. Business Query 7 — Highest-performing student in every batch.

```
SELECT BatchName, StudentName, AverageScore
FROM (
    SELECT B.BatchName, S.StudentName,
```

```

        AVG(R.Score) AS AverageScore,
        RANK() OVER (
            PARTITION BY B.BatchID
            ORDER BY AVG(R.Score) DESC
        ) AS rnk
    FROM Batch B
    JOIN Enrollment E ON B.BatchID = E.BatchID
    JOIN Student S ON E.StudentID = S.StudentID
    JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID
    GROUP BY B.BatchID, B.BatchName, S.StudentID, S.StudentName
) X
WHERE rnk = 1;

```

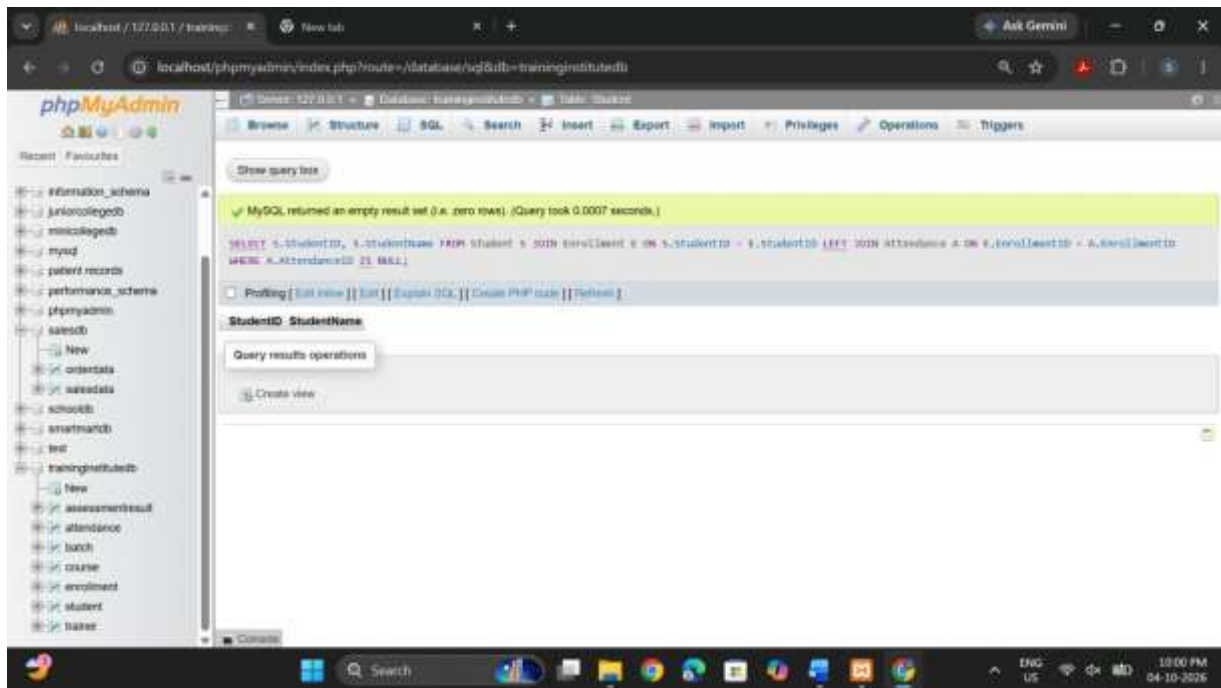


- 11. Business Query 8 — Students who have not attended any session.

```

SELECT S.StudentID, S.StudentName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
LEFT JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID
WHERE A.AttendanceID IS NULL;

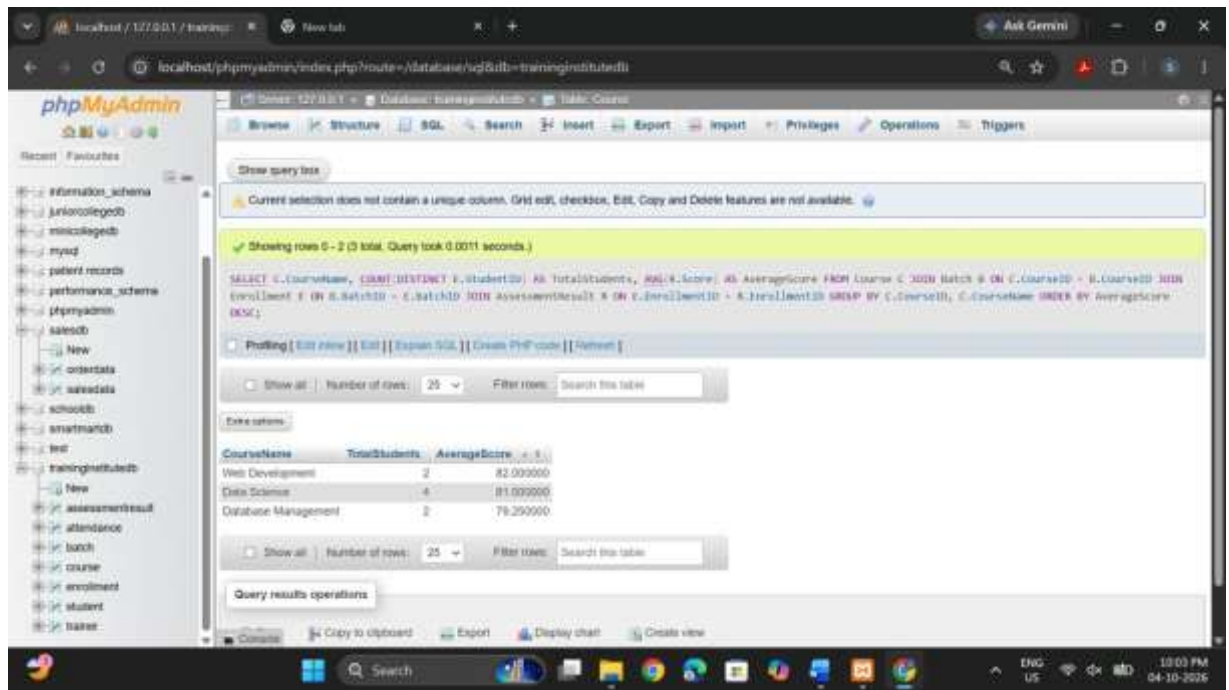
```



- With the supplied sample data, every enrolled student has attendance records, so this query returns no rows. This is useful for identifying students needing immediate follow-up.

12. Business Query 9 — Compare course-wise student performance.

```
SELECT C.CourseName,
       COUNT(DISTINCT E.StudentID) AS TotalStudents,
       AVG(R.Score) AS AverageScore
FROM Course C
JOIN Batch B ON C.CourseID = B.CourseID
JOIN Enrollment E ON B.BatchID = E.BatchID
JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID
GROUP BY C.CourseID, C.CourseName
ORDER BY AverageScore DESC;
```



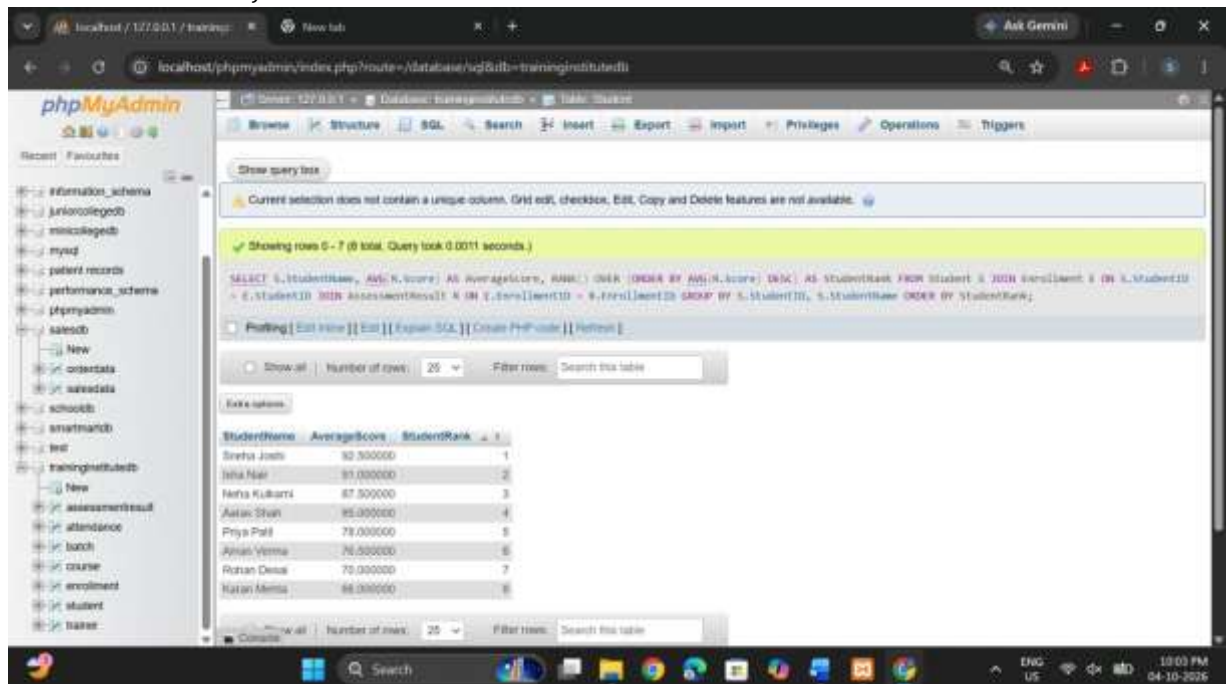
Showing rows 0 - 2 (3 total. Query took 0.0011 seconds.)

```
SELECT C.CourseName, COUNT(DISTINCT S.StudentID) AS TotalStudents, AVG(R.Score) AS AverageScore FROM Course C JOIN Batch B ON C.CourseID = B.CourseID JOIN Enrollment E ON B.BatchID = E.BatchID JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID GROUP BY C.CourseID, C.CourseName ORDER BY AverageScore DESC;
```

CourseName	TotalStudents	AverageScore
Web Development	2	82.000000
Data Science	4	81.000000
Database Management	2	78.250000

- 13. Business Query 10 — Rank students by assessment performance.

```
SELECT S.StudentName,
       AVG(R.Score) AS AverageScore,
       RANK() OVER (ORDER BY AVG(R.Score) DESC) AS StudentRank
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID
GROUP BY S.StudentID, S.StudentName
ORDER BY StudentRank;
```



Showing rows 0 - 7 (8 total. Query took 0.0011 seconds.)

```
SELECT S.StudentName, AVG(R.Score) AS AverageScore, RANK() OVER (ORDER BY AVG(R.Score) DESC) AS StudentRank FROM Student S JOIN Enrollment E ON S.StudentID = E.StudentID JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID GROUP BY S.StudentID, S.StudentName ORDER BY StudentRank;
```

StudentName	AverageScore	StudentRank
Sreha Joshi	82.500000	1
Isha Nair	81.000000	2
Neetha Kulkarni	81.500000	3
Aashan Shah	80.000000	4
Priya Patel	78.000000	5
Ayush Verma	76.000000	6
Rohan Desai	70.000000	7
Nutan Mehta	66.000000	8

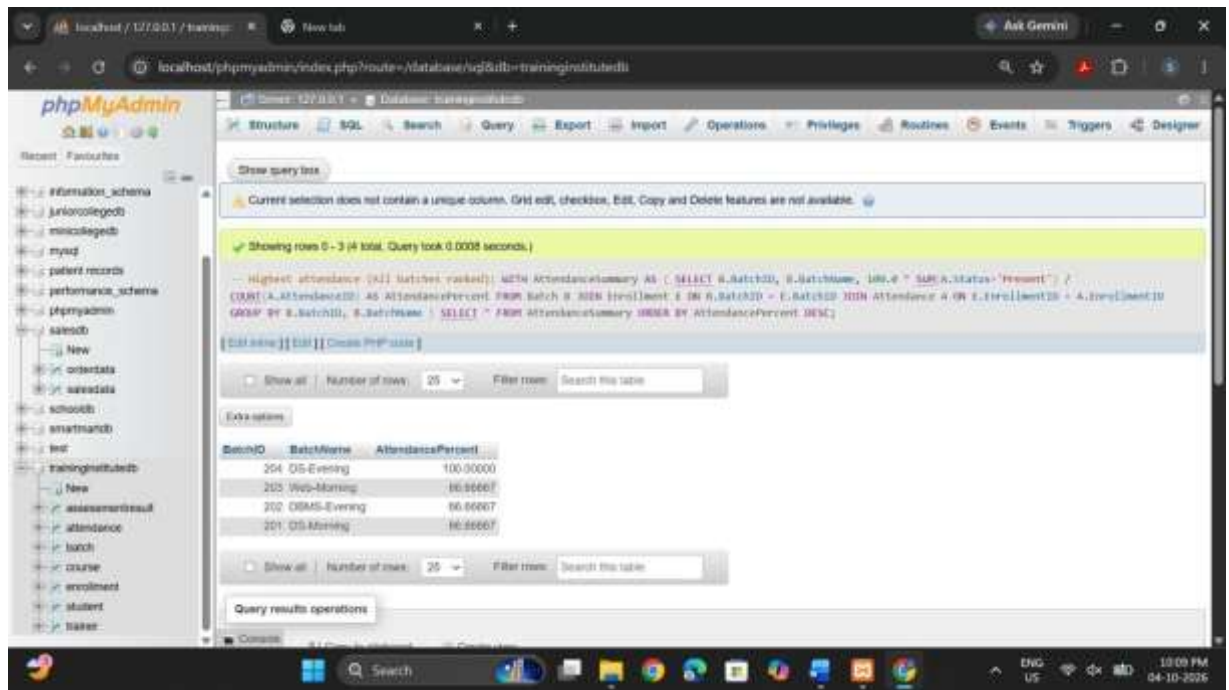
- 14. Business Query 11 — Batches with highest and lowest average attendance.

-- Highest attendance (All batches ranked):

```
WITH AttendanceSummary AS (  
    SELECT B.BatchID, B.BatchName,  
           100.0 * SUM(A.Status='Present') / COUNT(A.AttendanceID)  
           AS AttendancePercent  
    FROM Batch B  
    JOIN Enrollment E ON B.BatchID = E.BatchID  
    JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID  
    GROUP BY B.BatchID, B.BatchName  
)  
SELECT * FROM AttendanceSummary  
ORDER BY AttendancePercent DESC;
```

-- Lowest attendance:

```
WITH AttendanceSummary AS (  
    SELECT B.BatchID, B.BatchName,  
           100.0 * SUM(A.Status='Present') / COUNT(A.AttendanceID)  
           AS AttendancePercent  
    FROM Batch B  
    JOIN Enrollment E ON B.BatchID = E.BatchID  
    JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID  
    GROUP BY B.BatchID, B.BatchName  
)  
SELECT * FROM AttendanceSummary  
ORDER BY AttendancePercent ASC  
LIMIT 1;
```



• 15. Business Query 12 — Student-wise training progress report.

```

SELECT S.StudentName,
       B.BatchName,
       C.CourseName,
       ROUND(100.0 * SUM(A.Status='Present') / COUNT(A.AttendanceID),2)
         AS AttendancePercent,
       ROUND(AVG(R.Score),2) AS AverageScore
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Batch B ON E.BatchID = B.BatchID
JOIN Course C ON B.CourseID = C.CourseID
LEFT JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID
LEFT JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID
GROUP BY S.StudentID, S.StudentName, B.BatchName, C.CourseName
ORDER BY AverageScore DESC;

```

Showing rows 0 - 7 (8 total). Query took 0.0018 seconds. [AverageScore: 92.50... +66.08...]

```
SELECT S.StudentName, B.BatchName, C.CourseName, ROUND(SUM(B * IF(SUB(A.Status='Present')) / COUNT(A.AttendanceID),2) AS AttendancePercent,
ROUND(SUM(K.Score),2) AS AverageScore FROM Student S JOIN Enrollment E ON S.StudentID = E.StudentID JOIN Batch B ON S.BatchID = B.BatchID JOIN Course C ON
B.CourseID = C.CourseID LEFT JOIN Attendance A ON E.EnrollmentID = A.EnrollmentID LEFT JOIN AssessmentResult R ON E.EnrollmentID = R.EnrollmentID GROUP BY
S.StudentID, S.StudentName, B.BatchName, C.CourseName ORDER BY AverageScore DESC;
```

StudentName	BatchName	CourseName	AttendancePercent	AverageScore
Sneha Joshi	DBMS-Evening	Database Management	100.00	92.50
Neha Nair	DS-Evening	Data Science	100.00	91.00
Neha Kulkarni	Web-Morning	Web Development	100.00	87.50
Aarav Shah	DS-Morning	Data Science	100.00	85.00
Priya Patel	DS-Morning	Data Science	86.67	78.00
Aman Verma	Web-Morning	Web Development	33.33	76.50
Rohan Desai	DS-Morning	Data Science	33.33	70.00
Rohan Mehta	DBMS-Evening	Database Management	33.33	66.00

16. Project Insights

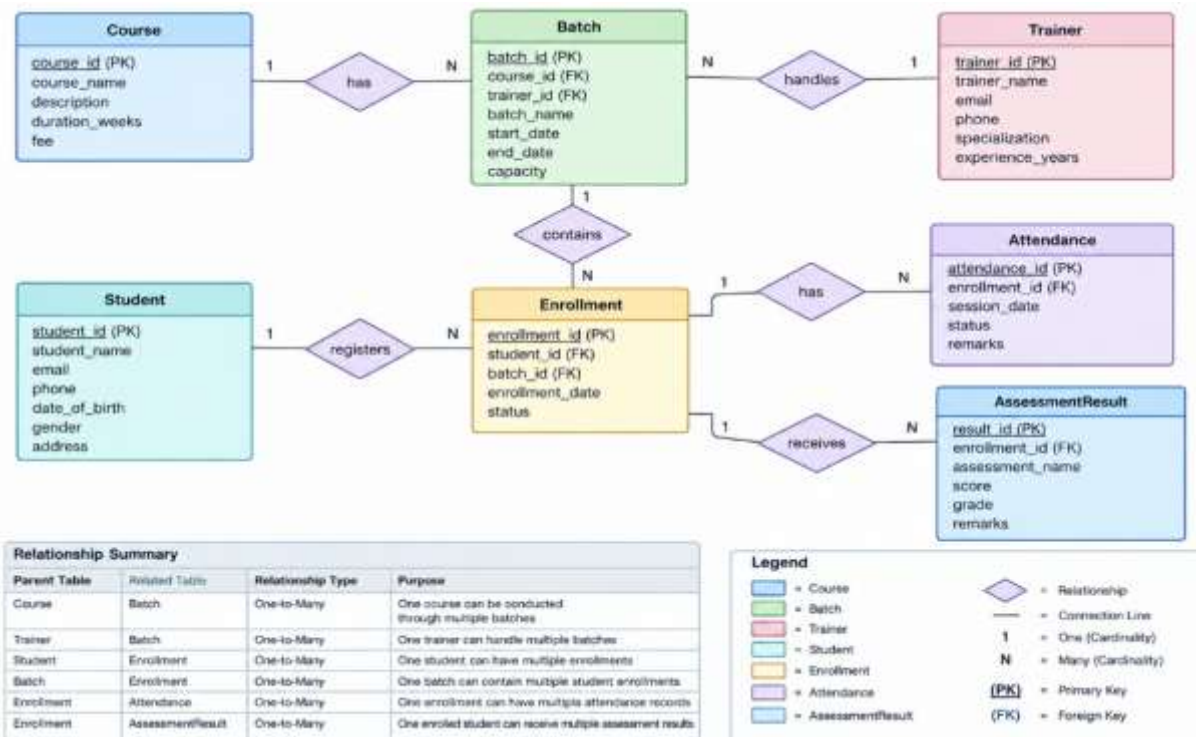
- The queries identify course enrollment, attendance risk, assessment performance, batch-level performance and student progress. The institute can use these results to contact students with low attendance, provide additional support to low scorers, recognize high performers and compare courses/batches. These are the analysis areas explicitly required by the workbook.

file:///C:/Users/turn2/Desktop/L76-L101/

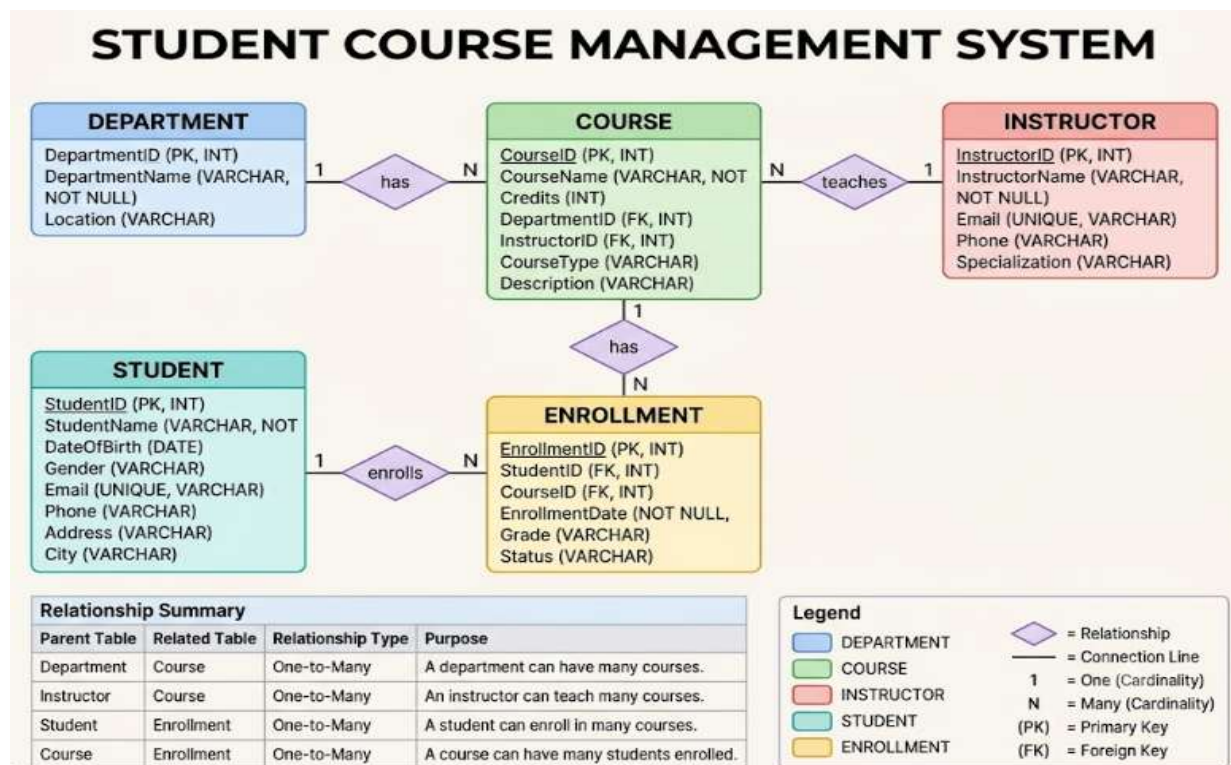
17. Conclusion

- The Training Institute Management System successfully organizes student, trainer, course, batch, enrollment, attendance and assessment data using primary and foreign keys. SQL joins, aggregate functions, subqueries, views and window functions can then be used to produce useful management reports. The system reduces repeated data, maintains relationships between records and supports data-driven decisions.

ER-Diagram



Schema Diagram



Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well Done []

Signature of Instructor: _____

End of Lab Workbook
